The CREATE MODEL statement for ARIMA_PLUS models

This document describes the `CREATE MODEL` statement for creating univariate time series models in BigQuery by using SQL. Alternatively, you can use the Google Cloud console user interface to create a model by using a UI (/bigquery/docs/create-machine-learning-model-console) (Preview (https://cloud.google.com/products#product-launch-stages)) instead of constructing the SQL statement yourself. You can use a univariate time series model to forecast (/bigquery/docs/forecasting-overview) the future value for a given column based on the analysis of historical values for that column. You can also use univariate time series models to detect anomalies in time series data.

- Use the `ML.FORECAST` function (/bigquery/docs/reference/standard-sql/bigqueryml-syntax-forecast) to retrieve the forecasted values that were generated when you created the model.
- Use the `ML.EXPLAIN_FORECAST` function (/bigquery/docs/reference/standard-sql/bigqueryml-syntax-explain-forecast) function to retrieve the forecasted values that were generated when you created the model, and compute the prediction intervals.
- Use the `ML.DETECT_ANOMALIES` function (/bigquery/docs/reference/standard-sql/bigqueryml-syntax-detect-anomalies) to perform anomaly detection (/bigquery/docs/anomaly-detection-overview).

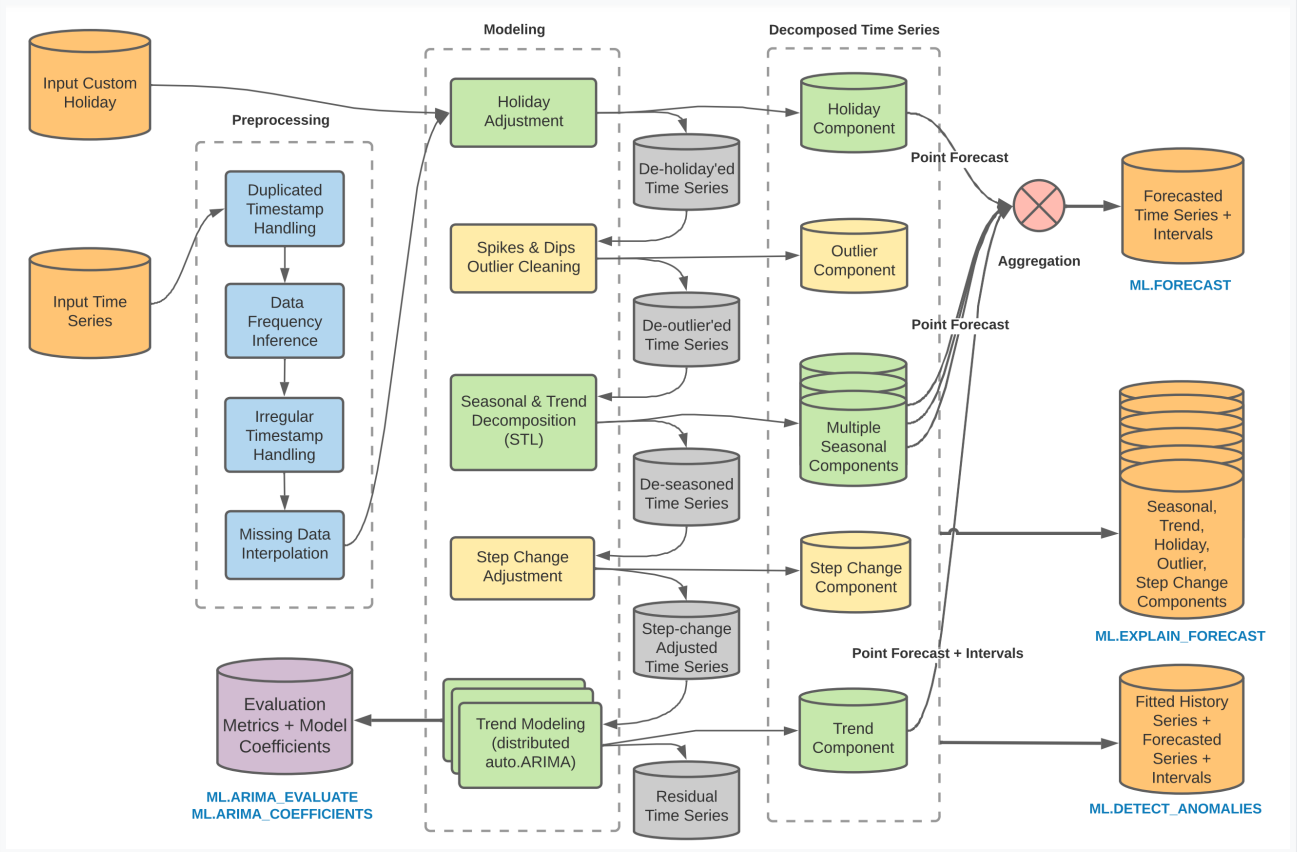
Note: If you don't want to create and manage your own model, you can use BigQuery ML's built-in TimesFM time series model (/bigquery/docs/timesfm-model) with the `AI.FORECAST` function (/bigquery/docs/reference/standard-sql/bigqueryml-syntax-ai-forecast) (Preview (https://cloud.google.com/products#product-launch-stages)) to perform forecasting.

For more information about supported SQL statements and functions for this model, see End-to-end user journeys for time series forecasting models (/bigquery/docs/e2e-journey-forecast).

Time series modeling pipeline

The BigQuery ML time series modeling pipeline includes multiple modules. The ARIMA (https://en.wikipedia.org/wiki/Autoregressive_integrated_moving_average) model is the most computationally expensive module, which is why the model is named `ARIMA_PLUS`. For

more information, see ARIMA_PLUS: Large-scale, Accurate, Automatic and Interpretable In-Database Time Series Forecasting and Anomaly Detection in Google BigQuery (https://arxiv.org/abs/2510.24452).



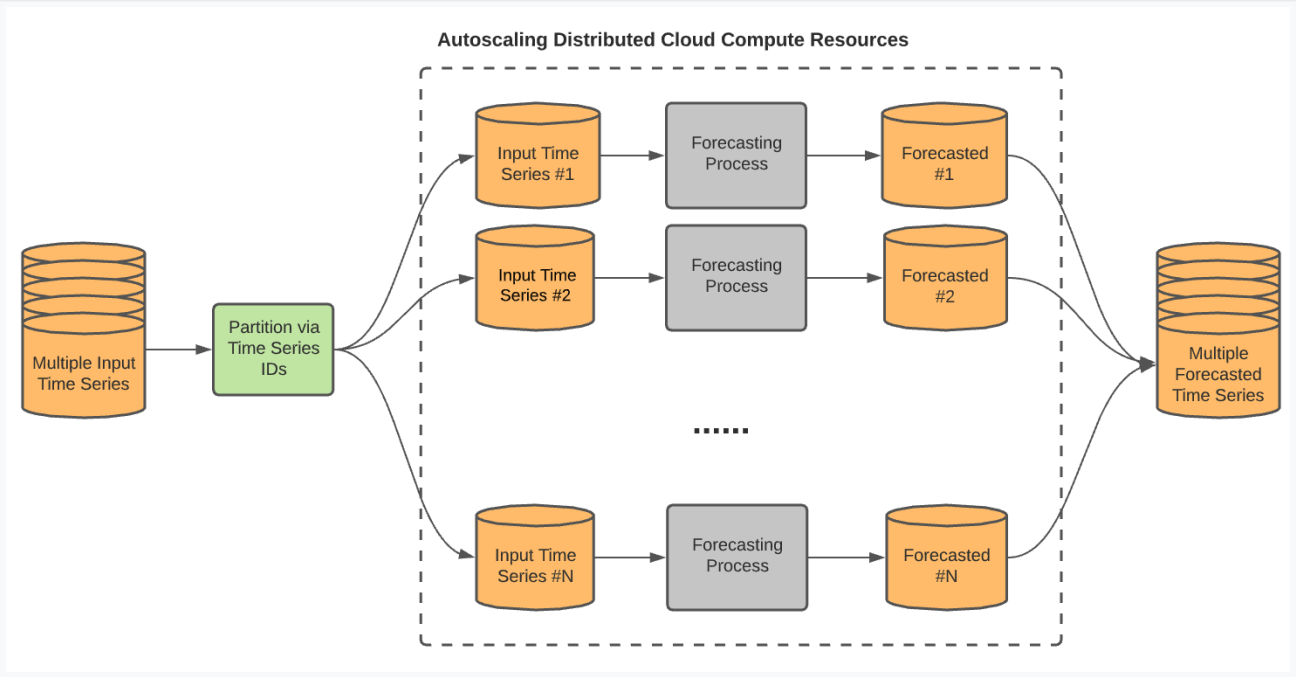
The modeling pipeline for the `ARIMA_PLUS` time series models performs the following functions:

- Infer the data frequency of the time series.
- Handle irregular time intervals.
- Handle duplicated timestamps by taking the mean value.
- Interpolate missing data using local linear interpolation.
- Detect and clean spike and dip outliers.
- Detect and adjust abrupt step (level) changes.
- Detect and adjust holiday effect.
- Detect multiple seasonal patterns within a single time series by using Seasonal and Trend decomposition using Loess (STL) (https://otexts.com/fpp2/stl.html), and extrapolate seasonality by using double exponential smoothing (ETS) (https://en.wikipedia.org/wiki/Exponential_smoothing#Double_exponential_smoothing).

- Detect and model the trend using the ARIMA model and the auto.ARIMA (https://otexts.com/fpp2/arima-r.html) algorithm for automatic hyperparameter tuning. In auto.ARIMA, dozens of candidate models are trained and evaluated in parallel. The model with the lowest Akaike information criterion (AIC) (https://en.wikipedia.org/wiki/Akaike_information_criterion) is selected as the best model.

Large-scale time series

You can forecast up to 100,000,000 time series simultaneously with a single query by using the `TIME_SERIES_ID_COL` (#time_series_id_col) option. With this option, different modeling pipelines run in parallel, as long as enough slots (/bigquery/docs/slots) are available. The following diagram shows this process:



Large-scale time series forecasting best practices

Forecasting many time series simultaneously can lead to long-running queries, because query processing isn't completely parallel due to limited slot capacity. The following best practices can help you avoid long-running queries when forecasting many time series simultaneously:

- When you have a large number (for example, 100,000) of time series to forecast, first forecast a small number of time series (for example, 1,000) to see how long the query takes. You can then estimate how long your entire time series forecast will take.

- You can use the `AUTO_ARIMA_MAX_ORDER` option to balance between query run time and forecast accuracy. Increasing `AUTO_ARIMA_MAX_ORDER` expands the hyperparameter search space to try more complex ARIMA models, that is, ARIMA models with higher non-seasonal p and q. This increases forecast accuracy but also increases query run time. Decreasing the value of `AUTO_ARIMA_MAX_ORDER` decreases forecast accuracy but also decreases query run time. For example, if you specify a value of `1` instead of using the `2` for this option, the query run time is reduced by more than 50%. The forecast accuracy might drop slightly for some of the time series. If a shorter training time is important to your use case, use a smaller value for `AUTO_ARIMA_MAX_ORDER`.
- The model training time for each time series has a linear relationship to its length, which is based on the number of data points. The longer the time series, the longer the training takes. However, not all data points contribute equally to the model fitting process. Instead, the more recent the data point is, the more it contributes to the process. Therefore, if you have a long time series, for example ten years of daily data, you don't need to train a time series model using all of the data points. The most recent two or three years of data points are enough.
- You can use the `TIME_SERIES_LENGTH_FRACTION`, `MIN_TIME_SERIES_LENGTH` and `MAX_TIME_SERIES_LENGTH` training options to enable fast model training with little to no loss of forecasting accuracy. The idea behind these options is that while periodic modeling, such as seasonality, requires a certain number of time points, trend modeling doesn't need many time points. However, trend modeling is much more computationally expensive than other time series components. By using the aforementioned training options, you can efficiently model the trend component with a subset of the time series, while the other time series components use the entire time series.
- To avoid a single long-running query, use BigQuery multi-statement queries (/bigquery/docs/multi-statement-queries).
- In some cases, your input table might be missing values in the feature columns. For example, in a multivariate time series with features A and B, most of the rows contain valid values for both A and B, while some might have `NULL` values for one or the other of the features. BigQuery ML fills in the missing values by using the mean value of the entire data column, which might include values from different time series. This causes discrepancies in the results when compared to single time series training, because the input values are affected by other time series. The best practice is to analyze the data and impute the missing feature values before running `CREATE MODEL`. For example, using the mean value of that feature within each time series, or assigning a value of zero.

You can try these best practices by following the Scalable forecasting with millions of time series in BigQuery (/bigquery/docs/arima-speed-up-tutorial) tutorial.

CREATE MODEL syntax

```
{CREATE MODEL (#create_model) | CREATE MODEL IF NOT EXISTS (#create_model_if_not_exists)
model_name (#model_name)
OPTIONS(model_option_list)
AS (#as) { query_statement (#query_statement) |
  (
    training_data AS (query_statement (#query_statement)),
    custom_holiday AS (holiday_statement (#holiday_statement))
  )
}

model_option_list:
MODEL_TYPE (#model_type) = 'ARIMA_PLUS'
    [, TIME_SERIES_TIMESTAMP_COL (#time_series_timestamp_col) = string_value ]
    [, TIME_SERIES_DATA_COL (#time_series_data_col) = string_value ]
    [, TIME_SERIES_ID_COL (#time_series_id_col) = { string_value | string_array } ]
    [, HORIZON (#horizon) = int64_value ]
    [, AUTO_ARIMA (#auto_arima) = { TRUE | FALSE } ]
    [, AUTO_ARIMA_MAX_ORDER (#auto_arima_max_order) = int64_value ]
    [, AUTO_ARIMA_MIN_ORDER (#auto_arima_min_order) = int64_value ]
    [, NON_SEASONAL_ORDER (#non_seasonal_order) = (int64_value, int64_value, int64
    [, DATA_FREQUENCY (#data_frequency) = { 'AUTO_FREQUENCY' | 'PER_MINUTE' | 'HOU
    [, INCLUDE_DRIFT (#include_drift) = { TRUE | FALSE } ]
    [, HOLIDAY_REGION (#holiday_region) = string_value | string_array ]
    [, CLEAN_SPIKES_AND_DIPS (#clean_spikes_and_dips) = { TRUE | FALSE } ]
    [, ADJUST_STEP_CHANGES (#adjust_step_changes) = { TRUE | FALSE } ]
    [, TIME_SERIES_LENGTH_FRACTION (#time_series_length_fraction) = float64_value ]
    [, MIN_TIME_SERIES_LENGTH (#min_time_series_length) = int64_value ]
    [, MAX_TIME_SERIES_LENGTH (#max_time_series_length) = int64_value ]
    [, TREND_SMOOTHING_WINDOW_SIZE (#trend_smoothing_window_size) = int64_value ]
    [, DECOMPOSE_TIME_SERIES (#decompose_time_series) = { TRUE | FALSE } ]
    [, FORECAST_LIMIT_LOWER_BOUND (#forecast_limit_lower_bound) = float64_value ]
    [, FORECAST_LIMIT_UPPER_BOUND (#forecast_limit_upper_bound) = float64_value ]
    [, SEASONALITIES (#seasonalities) = string_array ]
    [, HIERARCHICAL_TIME_SERIES_COLS (#hierarchical_time_series_cols) = {string_array
    [, KMS_KEY_NAME (#kms_key_name) = string_value ]
```

`CREATE MODEL`

Creates and trains a new model in the specified dataset. If the model name exists, `CREATE MODEL` returns an error.

`CREATE MODEL IF NOT EXISTS`

Creates and trains a new model only if the model doesn't exist in the specified dataset.

`CREATE OR REPLACE MODEL`

Creates and trains a model and replaces an existing model with the same name in the specified dataset.

`model_name`

The name of the model you're creating or replacing. The model name must be unique in the dataset: no other model or table can have the same name. The model name must follow the same naming rules as a BigQuery table. A model name can:

- Contain up to 1,024 characters
- Contain letters (upper or lower case), numbers, and underscores

`model_name` is case-sensitive.

If you don't have a default project configured, then you must prepend the project ID to the model name in the following format, including backticks:

`[PROJECT_ID].[DATASET].[MODEL]`

For example, `myproject.mydataset.mymodel`.

`MODEL_TYPE`

Syntax

```
MODEL_TYPE = 'ARIMA_PLUS'
```

Description

Specifies the model type. This option is required.

> **Note:** The **`ARIMA`** model type is deprecated. While the model training pipelines of **`ARIMA`** and **`ARIMA_PLUS`** models are the same, **`ARIMA_PLUS`** supports more capabilities, including use of the **`DECOMPOSE_TIME_SERIES`** option (/bigquery/docs/reference/standard-sql/bigqueryml-syntax-create-time-series#decompose_time_series) and the ability to work with the **`ML.ARIMA_EVALUATE`** (/bigquery/docs/reference/standard-sql/bigqueryml-syntax-arima-evaluate) and **`ML.EXPLAIN_FORECAST`** (/bigquery/docs/reference/standard-sql/bigqueryml-syntax-explain-forecast) functions.

`TIME_SERIES_TIMESTAMP_COL`

Syntax

```
TIME_SERIES_TIMESTAMP_COL = string_value
```

Description

The name of the column that provides the time points used in training the model. The column must be of one of the following data types:

- `TIMESTAMP`
- `DATE`
- `DATETIME`

Arguments

A `STRING` value.

`TIME_SERIES_DATA_COL`

Syntax

```
TIME_SERIES_DATA_COL = string_value
```

Description

The name of the column that contains the data to forecast. The column must be of one of the following data types:

- `INT64`

- `NUMERIC`
- `BIGNUMERIC`
- `FLOAT64`

Arguments

A `STRING` value.

`TIME_SERIES_ID_COL`

Syntax

```
TIME_SERIES_ID_COL = { string_value | string_array }
```

Description

The names of the ID columns. Specify one or more values for this option when you want to fit and forecast multiple time series using a single query. Each ID identifies a unique time series. The columns must be of one of the following data types:

- `STRING`
- `INT64`
- `ARRAY<STRING>`
- `ARRAY<INT64>`

Arguments

A `STRING` or `ARRAY<STRING>` value.

`HORIZON`

Syntax

```
HORIZON = int64_value
```

Description

The number of time points to forecast.

When forecasting multiple time series at once, this parameter applies to each time series.

Arguments

An `INT64` value. The default value is `1,000`. The maximum value is `10,000`.

AUTO_ARIMA

Syntax

```
AUTO_ARIMA = { TRUE | FALSE }
```

Description

Determines whether the training process uses auto.ARIMA or not. If `TRUE`, training automatically finds the best non-seasonal order (that is, the p, d, q tuple) and decides whether or not to include a linear drift term when d is 1. If `FALSE`, you must specify the `NON_SEASONAL_ORDER` option.

When forecasting multiple time series at the same time, you must use the auto.ARIMA algorithm for each time series, so this option must be `TRUE`.

Arguments

A `BOOL` value. The default value is `TRUE`.

AUTO_ARIMA_MAX_ORDER

Syntax

```
AUTO_ARIMA_MAX_ORDER = int64_value
```

Description

The maximum value for the sum of non-seasonal p and q. This value determines the parameter search space in the auto.ARIMA algorithm, in combination with the `AUTO_ARIMA_MIN_ORDER` value. This option is disabled when the `AUTO_ARIMA` value is `FALSE`.

Arguments

An `INT64` value between `1` and `5`, inclusive. The default value is `2`.

If non-seasonal d is determined to be 0 or 2, the number of candidate models evaluated for each supported value is as follows:

- `1`: 3 candidate models
- `2`: 6 candidate models
- `3`: 10 candidate models
- `4`: 15 candidate models
- `5`: 21 candidate models

If non-seasonal d is determined to be 1, the number of candidate models to evaluate is doubled, because there's an additional drift term to consider for all of the existing candidate models.

> **Note:** the number of bytes processed by the input **`SELECT`** statement is multiplied by the number of candidate models, which is controlled by the **`AUTO_ARIMA_MAX_ORDER`** and **`AUTO_ARIMA_MIN_ORDER`** options. This affects the pricing. See BigQuery ML pricing (https://cloud.google.com/bigquery/pricing#bqml) for details.

`AUTO_ARIMA_MIN_ORDER`

Syntax

```
AUTO_ARIMA_MIN_ORDER = int64_value
```

Description

The minimum value for the sum of non-seasonal p and q. This value determines the parameter search space in the auto.ARIMA algorithm, in combination with the `AUTO_ARIMA_MAX_ORDER` value. Setting this option to `1` or greater lets the model exclude some flat forecasting results. This option is disabled when `AUTO_ARIMA` is `FALSE`.

Arguments

The value is a `INT64`. The default value is `0`.

> **Note:** If **`AUTO_ARIMA_MAX_ORDER`** is set, the **`AUTO_ARIMA_MIN_ORDER`** value must be less than the **`AUTO_ARIMA_MAX_ORDER`** value.

`NON_SEASONAL_ORDER`

Syntax

```
NON_SEASONAL_ORDER = (p_value, d_value, q_value)
```

Description

The tuple of non-seasonal p, d, q for the `ARIMA_PLUS` model. There are no default values, and you must specify all three values. p and q must be a value between `0` and `5`, inclusive. d must be a value between `0` and `2`, inclusive.

You can't use this option when forecasting multiple time series at the same time, because the auto.ARIMA algorithm must be used for each time series.

The `AUTO_ARIMA` value must be `FALSE` to use this option.

Arguments

A tuple of three `INT64` values. For example, `(1, 2, 1)`.

DATA_FREQUENCY

Syntax

```
DATA_FREQUENCY = { 'AUTO_FREQUENCY' | 'PER_MINUTE' | 'HOURLY' | 'DAILY' | 'WEE
```

Description

The data frequency of the input time series. The finest supported granularity is `PER_MINUTE`.

When forecasting multiple time series at once, this argument applies to all individual time series.

Arguments

This option accepts the following values:

- `AUTO_FREQUENCY`: This is the default. The training process automatically infers the data frequency, which can be any of the other supported values for this option.
- `PER_MINUTE`
- `HOURLY`
- `DAILY`
- `WEEKLY`

- `MONTHLY`
- `QUARTERLY`
- `YEARLY`

INCLUDE_DRIFT

Syntax

```
INCLUDE_DRIFT = { TRUE | FALSE }
```

Description

Determines whether the `ARIMA_PLUS` model should include a linear drift term or not. The drift term is applicable when non-seasonal d is 1.

- When the `AUTO_ARIMA` value is `FALSE` , this argument defaults to `FALSE`. You can set it to `TRUE` only when non-seasonal d is 1. Otherwise the `CREATE MODEL` statement returns an invalid query error.
- When the `AUTO_ARIMA` value is `TRUE`, BigQuery ML automatically determines whether or not to include a linear drift term, so you can't use this option.

Arguments

A `BOOL` value. The default value is `FALSE`.

HOLIDAY_REGION

Syntax

```
HOLIDAY_REGION = string_value | string_array
```

Description

The geographical region based on which the holiday effect is applied in modeling. By default, holiday effect modeling isn't used. To use it, specify one or more holiday regions using this option. If you include more than one region string, the union of the holidays in all the provided regions are taken into account when modeling.

Holiday effect modeling is only applicable when the time series is daily or weekly, and longer than a year. If the input time series doesn't meet these requirements, holiday effect modeling isn't used even if you specify this option.

For more information about the holidays included in each region, see Holiday data (#holiday_data).

Arguments

A `STRING` or `ARRAY<STRING>` value.

Use a single string value to identify one region. For example:

```
HOLIDAY_REGION = 'GLOBAL'
```

Use an array of string values to identify multiple regions. For example:

```
HOLIDAY_REGION = ['US', 'GB']
```

This option accepts the following values:

Global

- `GLOBAL`

Continental regions

- `NA`: North America
- `JAPAC`: Japan and Asia Pacific
- `EMEA`: Europe, the Middle East and Africa
- `LAC`: Latin America and the Caribbean

Countries

- `AE`: United Arab Emirates
- `AR`: Argentina
- `AT`: Austria

- `AU`: Australia
- `BE`: Belgium
- `BR`: Brazil
- `CA`: Canada
- `CH`: Switzerland
- `CL`: Chile
- `CN`: China
- `CO`: Colombia
- `CZ`: Czechia
- `DE`: Germany
- `DK`: Denmark
- `DZ`: Algeria
- `EC`: Ecuador
- `EE`: Estonia
- `EG`: Egypt
- `ES`: Spain
- `FI`: Finland
- `FR`: France
- `GB`: United Kingdom
- `GR`: Greece
- `HK`: Hong Kong
- `HU`: Hungary
- `ID`: Indonesia
- `IE`: Ireland
- `IL`: Israel
- `IN`: India
- `IR`: Iran

- `IT`: Italy
- `JP`: Japan
- `KR`: South Korea
- `LV`: Latvia
- `MA`: Morocco
- `MX`: Mexico
- `MY`: Malaysia
- `NG`: Nigeria
- `NL`: Netherlands
- `NO`: Norway
- `NZ`: New Zealand
- `PE`: Peru
- `PH`: Philippines
- `PK`: Pakistan
- `PL`: Poland
- `PT`: Portugal
- `RO`: Romania
- `RS`: Serbia
- `RU`: Russia
- `SA`: Saudi Arabia
- `SE`: Sweden
- `SG`: Singapore
- `SI`: Slovenia
- `SK`: Slovakia
- `TH`: Thailand
- `TR`: Turkey
- `TW`: Taiwan

- `UA`: Ukraine
- `US`: United States
- `VE`: Venezuela
- `VN`: Vietnam
- `ZA`: South Africa

CLEAN_SPIKES_AND_DIPS

Syntax

```
CLEAN_SPIKES_AND_DIPS = { TRUE | FALSE }
```

Description

Determines whether or not to perform automatic spikes and dips detection and cleanup in the `ARIMA_PLUS` model training pipeline. The spikes and dips are replaced with local linear interpolated values when they're detected.

Arguments

A `BOOL` value. The default value is `TRUE`.

ADJUST_STEP_CHANGES

Syntax

```
ADJUST_STEP_CHANGES = { TRUE | FALSE }
```

Description

Determines whether or not to perform automatic step change detection and adjustment in the `ARIMA_PLUS` model training pipeline.

Arguments

A `BOOL` value. The default value is `TRUE`.

TIME_SERIES_LENGTH_FRACTION

Syntax

```
TIME_SERIES_LENGTH_FRACTION = float64_value
```

Description

The fraction of the interpolated length of the time series that's used to model the time series trend component. All of the time points of the time series are used to model the non-trend component. For example, if the time series has 100 time points, then specifying a `TIME_SERIES_LENGTH_FRACTION` of `0.5` uses the most recent 50 time points for trend modeling. This training option accelerates modeling training without sacrificing much forecasting accuracy.

You can use the `TIME_SERIES_LENGTH_FRACTION` option with the `MIN_TIME_SERIES_LENGTH` option, but not with the `MAX_TIME_SERIES_LENGTH` option.

Arguments

A `FLOAT64` value in the range `(0, 1)`. The default behavior is to use all the points in the time series.

MIN_TIME_SERIES_LENGTH

Syntax

```
MIN_TIME_SERIES_LENGTH = int64_value
```

Description

The minimum number of time points that are used in modeling the trend component of the time series. If you use this option, you must also specify a value for the `TIME_SERIES_LENGTH_FRACTION` option, and you can't specify a value for the `MAX_TIME_SERIES_LENGTH` option.

For example, if you use `TIME_SERIES_ID_COL` to forecast two time series, one with 100 time points and another with 30 time points, then setting `TIME_SERIES_LENGTH_FRACTION` to `0.5` and `MIN_TIME_SERIES_LENGTH` to `20` results in the last 50 points of first time series being used for trend modeling. For the second time series, the last 20 points rather than the last 15 points (`30 * 0.5`) are used in trend modeling because the `MIN_TIME_SERIES_LENGTH` value is `20`. This option ensures that enough time points are available when you use `TIME_SERIES_LENGTH_FRACTION` in trend modeling. This is particularly important when forecasting multiple time series in a single query using the

`TIME_SERIES_ID_COL` option. If the total number of time points is less than the `MIN_TIME_SERIES_LENGTH` value, then the query uses all available time points.

Arguments

An `INT64` value greater than or equal to `4`. The default value is `20`.

MAX_TIME_SERIES_LENGTH

Syntax

```
MAX_TIME_SERIES_LENGTH = int64_value
```

Description

The maximum number of time points in a time series that can be used in modeling the trend component of the time series.

For example, if you are forecasting two time series simultaneously by specifying the `TIME_SERIES_ID_COL` option, and one time series has 100 time points while the other one has 50 time points, then by setting `MAX_TIME_SERIES_LENGTH` to `30`, both of the time series use the last 30 time points for trend modeling.

You can't use the`MAX_TIME_SERIES_LENGTH` with the `TIME_SERIES_LENGTH_FRACTION` or `MIN_TIME_SERIES_LENGTH` options.

Arguments

An `INT64` value greater than or equal to `4`. There is no default value. We recommend trying `30` as a starting value.

TREND_SMOOTHING_WINDOW_SIZE

Syntax

```
TREND_SMOOTHING_WINDOW_SIZE = int64_value
```

Description

The smoothing window size for the trend component. When you specify a value, a center moving average smoothing is applied on the history trend. When the smoothing window is out of the boundary at the beginning or the end of the trend, the first element or the last element is padded to fill the smoothing window before the average is applied.

Specifying a value for `TREND_SMOOTHING_WINDOW_SIZE` doesn't affect forecasting results. It only affects the smoothness of the trend component, which you can see by using the `ML.EXPLAIN_FORECAST` function (/bigquery/docs/reference/standard-sql/bigqueryml-syntax-explain-forecast).

Arguments

An `INT64` value. There is no default value. You must specify a positive value to smooth the trend.

DECOMPOSE_TIME_SERIES

Syntax

```
DECOMPOSE_TIME_SERIES = { TRUE | FALSE }
```

Description Determines whether the separate components of both the history and forecast parts of the time series (such as holiday effect and seasonal components) are saved in the `ARIMA_PLUS` model.

Time series decomposition takes place when you create the model. The `ML.EXPLAIN_FORECAST` (/bigquery/docs/reference/standard-sql/bigqueryml-syntax-explain-forecast) function retrieves the separate components of both the training and the forecasting data and computes the confidence intervals. Because the decomposition results are saved in the model, the training data can be partially or fully recovered from the decomposition results.

Arguments

A `BOOL` value. The default value is `TRUE`.

FORECAST_LIMIT_LOWER_BOUND

Syntax

`FORECAST_LIMIT_LOWER_BOUND = float64_value`

Description

The lower bound of the forecasting values. When you specify the `FORECAST_LIMIT_LOWER_BOUND` option, all of the forecast values must be greater than the specified value. For example, if you set `FORECAST_LIMIT_LOWER_BOUND` to `0`, then all of the

forecast values are larger than `0`. Also, all values less than or equal to the `FORECAST_LIMIT_LOWER_BOUND` value are excluded from modelling. The forecasting limit ensures that forecasts stay within limits (https://otexts.com/fpp2/limits.html).

If you specify a value for the`FORECAST_LIMIT_UPPER_BOUND` option, the `FORECAST_LIMIT_UPPER_BOUND` value must be greater than the `FORECAST_LIMIT_LOWER_BOUND` value.

Arguments

A `FLOAT_64` value greater than or equal to `-1.7976931348623157E+308`.

FORECAST_LIMIT_UPPER_BOUND

Syntax

```
FORECAST_LIMIT_UPPER_BOUND = float64_value
```

Description

The upper bound of the forecasting values. When you specify the `FORECAST_LIMIT_UPPER_BOUND` option, all of the forecast values must be less than the specified value. For example, if you set `FORECAST_LIMIT_UPPER_BOUND` to `100`, then all of the forecast values are less than `100`. Also, all values greater than or equal to the `FORECAST_LIMIT_UPPER_BOUND` value are excluded from modelling. The forecasting limit ensures that forecasts stay within limits (https://otexts.com/fpp2/limits.html).

If you specify a value for the`FORECAST_LIMIT_LOWER_BOUND` option, the `FORECAST_LIMIT_LOWER_BOUND` value must be less than the `FORECAST_LIMIT_UPPER_BOUND` value.

Arguments

A `FLOAT_64` value less thanor equal to `1.7976931348623157E+308`.

SEASONALITIES

Syntax

```
SEASONALITIES = string_array
```

Description

The seasonality of the time series data refers to the presence of variations that occur at certain regular intervals such as weekly, monthly or quarterly. Specifying the seasonality helps the model more accurately learn and predict the cyclic trends in your data. The input seasonality you provide as an argument is ignored if it's more granular than the finest seasonal granularity detected in the time series data. For example, if you input `['DAILY', 'WEEKLY', 'MONTHLY']` for this option, but input time series contains weekly data, then the `DAILY` variable is ignored during the model training.

Arguments

An `ARRAY<STRING>` value. The following string values are accepted:

- `AUTO`: This is the default. The training process automatically infers the seasonalities by data frequency.
- `NO_SEASONALITY`: Deactivates automatic seasonality detection.
- `DAILY`
- `WEEKLY`
- `MONTHLY`
- `QUARTERLY`
- `YEARLY`

You can only use the `NO_SEASONALITY` or `AUTO` values by themselves. For example, `['NO_SEASONALITY', 'DAILY']` isn't a valid value for this option.

HIERARCHICAL_TIME_SERIES_COLS

Syntax

```
HIERARCHICAL_TIME_SERIES_COLS = { string_array }
```

Description

The column names used to generate hierarchical time series forecasts. Specify one or more values for this option to aggregate and roll up values for all time series. The column order represents the hierarchy structure, where the left-most column is the parent. The columns must be of one of the following data types:

- `STRING`
- `INT64`

Arguments

An `ARRAY<STRING>` value.

KMS_KEY_NAME

Syntax

`KMS_KEY_NAME = string_value`

Description

The Cloud Key Management Service customer-managed encryption key (CMEK) (/kms/docs/cmek) to use to encrypt the model.

Arguments

A `STRING` value containing the fully-qualified name of the CMEK. For example,

```
'projects/my_project/locations/my_location/keyRings/my_ring/cryptoKeys/my_key
```

AS

All time series forecasting models support the following `AS` clause syntax for specifying the training data:

```
AS query_statement (#query_statement)
```

For time series forecasting models that have a `DATA_FREQUENCY` value of either `DAILY` or `AUTO_FREQUENCY`, you can optionally use the following `AS` clause syntax to perform custom holiday modeling (/bigquery/docs/reference/standard-sql/bigqueryml-syntax-create-time-series#custom_holidays) in addition to specifying the training data:

```
AS (
  training_data AS (query_statement (#query_statement)),
```

```
  custom_holiday AS (holiday_statement (#holiday_statement))
)
```

query_statement

The **query_statement** argument specifies the query that is used to generate the training data. For information about the supported SQL syntax of the **query_statement** clause, see GoogleSQL query syntax (/bigquery/docs/reference/standard-sql/query-syntax#sql_syntax).

holiday_statement

The **holiday_statement** argument specifies the query that provides custom holiday modeling information for time series forecast models. This query must return 50,000 rows or less and must contain the following columns:

- **region**: Required. A **STRING** value that identifies the region to target for holiday modeling. Use one of the following options:
  - An upper-case holiday region code (/bigquery/docs/reference/standard-sql/bigqueryml-syntax-create-time-series#holiday_region) .
    Use this option to overwrite or supplement the holidays for the specified region. You can see the holidays for a region by running **SELECT * FROM bigquery-public-data.ml_datasets.holidays_and_events_for_forecasting WHERE region = region**.
  - An arbitrary string. Use this option to specify a custom region that you want to model holidays for. For example, you could specify **London** if you are only modeling holidays for that city.

  Be sure not to use an existing holiday region code when you are trying to model for a custom region. For example, if you want to model a holiday in California, and specify **CA** as the **region** value, the service recognizes that as the holiday region code for Canada and targets that region. Because the argument is case-sensitive, you could specify **ca**, **California**, or some other value that isn't a holiday region code.
- **holiday_name**: Required. A **STRING** value that identifies the holiday to target for holiday modeling. Use one of the following options:
  - The holiday name as it is represented in the **bigquery-public-data.ml_datasets.holidays_and_events_for_forecasting** public table, including case. Use this option to overwrite

(#change_the_metadata_for_built-in_holidays) or supplement (#supplement_built-in_holidays_with_additional_custom_holidays) the specified holiday.

- A string that represents a custom holiday. The string must be a valid column name so that it can be used in `ML.EXPLAIN_FORECAST` output. For example, it cannot contain space. For more information on column naming, see Column names (/bigquery/docs/schemas#column_names).

- `primary_date`: Required. A `DATE` value that specifies the date the holiday falls on.
- `preholiday_days`: Optional. An `INT64` value that specifies the start of the holiday window around the holiday that is taken into account when modeling. Must be greater than or equal to `1`. Defaults to `1`.
- `postholiday_days`: Optional. An `INT64` value that specifies the end of the holiday window around the holiday that is taken into account when modeling. Must be greater than or equal to `1`. Defaults to `1`.

The `preholiday_days` and `postholiday_days` arguments together describe the holiday window around the holiday that is taken into account when modeling. The holiday window is defined as `[primary_date - preholiday_days, primary_date + postholiday_days]` and is inclusive of the pre- and post-holiday days. The value for each holiday window must be less than or equal to `30` and must be the same across the given holiday. For example, if you are modeling Arbor Day for several different years, you must specify the same holiday window for all of those years.

To achieve the best holiday modeling result, provide as much historical and forecast information about the occurrences of each included holiday as possible. For example, if you have time series data from 2018 to 2022 and would like to forecast for 2023, you get the best result by providing the custom holiday information for all of those years, similar to the following:

```
CREATE OR REPLACE MODEL `mydataset.arima_model`
  OPTIONS (
    model_type = 'ARIMA_PLUS',
    holiday_region = 'US',...) AS (
        training_data AS (SELECT * FROM `mydataset.timeseries_data`),
        custom_holiday AS (
            SELECT
              'US' AS region,
              'Halloween' AS holiday_name,
              primary_date,
              5 AS preholiday_days,
              1 AS postholiday_days
```

```
            FROM
              UNNEST(
                [
                  DATE('2018-10-31'),
                  DATE('2019-10-31'),
                  DATE('2020-10-31'),
                  DATE('2021-10-31'),
                  DATE('2022-10-31'),
                  DATE('2023-10-31')])
                AS primary_date
          )
      )
```

Holiday data

When you perform holiday modeling by specifying the `HOLIDAY_REGION` option, the model uses holiday data from the region or regions you specify. For example, the following table describes the holiday data used in the `US` region for the year 2022-2023.

- `region` specifies the geographic region to which the holiday applies. The supported regions are listed in `HOLIDAY_REGION` (#holiday_region).
- `holiday_name` contains the name of the holiday.
- `primary_date` specifies the date of the holiday. For holidays that span multiple days, this is usually the first day of the holiday.
- `preholiday_days` describes the number of days the holiday effect starts before the `primary_date` value.
- `postholiday_days` describes the number of days the holiday effect ends after the `primary_date` value.

region	holiday_name	primary_date	preholiday_days	postholiday_days
US	Christmas	2022-12-25	10	1
US	Christmas	2023-12-25	10	1
US	MothersDay	2022-05-08	6	1
US	MothersDay	2023-05-14	6	1
US	NewYear	2022-01-01	5	3

region	holiday_name	primary_date	preholiday_days	postholiday_days
US	NewYear	2023-01-01	5	3
US	DaylightSavingEnd	2022-11-06	1	1
US	DaylightSavingEnd	2023-11-05	1	1
US	DaylightSavingStart	2022-03-13	1	1
US	DaylightSavingStart	2023-03-12	1	1
US	Thanksgiving	2022-11-24	3	5
US	Thanksgiving	2023-11-23	3	5
US	Valentine	2022-02-14	3	1
US	Valentine	2023-02-14	3	1
US	EasterMonday	2022-04-18	8	1
US	EasterMonday	2023-04-10	8	1
US	Halloween	2022-10-31	1	1
US	Halloween	2023-10-31	1	1
US	StPatrickDay	2022-03-17	1	1
US	StPatrickDay	2023-03-17	1	1
US	ColumbusDay	2022-10-10	1	1
US	ColumbusDay	2023-10-09	1	1
US	IndependenceDay	2022-07-04	1	1
US	IndependenceDay	2023-07-04	1	1
US	Juneteenth	2022-06-19	1	1
US	Juneteenth	2023-06-19	1	1
US	LaborDay	2022-09-05	1	1
US	LaborDay	2023-09-04	1	1
US	MemorialDay	2022-05-30	1	1
US	MemorialDay	2023-05-29	1	1

region	holiday_name	primary_date	preholiday_days	postholiday_days
US	MLKDay	2022-01-17	1	1
US	MLKDay	2023-01-16	1	1
US	PresidentDay	2022-02-21	1	1
US	PresidentDay	2023-02-20	1	1
US	Superbowl	2022-02-13	1	1
US	Superbowl	2023-02-05	1	1
US	VeteranDay	2022-11-11	1	1
US	VeteranDay	2023-11-11	1	1

You can also see the holidays for a region by running `SELECT * FROM bigquery-public-data.ml_datasets.holidays_and_events_for_forecasting WHERE region = region`.

The `bigquery-public-data.ml_datasets.holidays_and_events_for_forecasting` table only contains holidays and events from the following regions:

- `AU`: Australia
- `CA`: Canada
- `CH`: Switzerland
- `CL`: Chile
- `CZ`: Czechia
- `DE`: Germany
- `DK`: Denmark
- `EMEA`: Europe, the Middle East and Africa
- `ES`: Spain
- `FR`: France
- `GB`: United Kingdom
- `GLOBAL`
- `ID`: Indonesia
- `IN`: India

- `IT`: Italy
- `JAPAC`: Japan and Asia Pacific
- `JP`: Japan
- `KR`: South Korea
- `LAC`: Latin America and the Caribbean
- `MX`: Mexico
- `MY`: Malaysia
- `NA`: North America
- `NL`: Netherlands
- `NZ`: New Zealand
- `PT`: Portugal
- `SK`: Slovakia
- `US`: United States
- `ZA`: South Africa

Custom holidays

You can combine use of the **`holiday_statement`** argument (#holiday_statement) and the **`HOLIDAY_REGION`** option (#holiday_region) to enable several different custom holiday scenarios, as described in the following sections.

Supplement built-in holidays with additional custom holidays

To model one or more custom holidays in addition to a region's built-in holidays, specify the target holiday region with the **`HOLIDAY_REGION`** option, and then provide the new holiday metadata in the **`holiday_statement`** argument.

The following example models all built-in holidays for the **`US`** holiday region, and additionally models the custom holiday **`members_day`**:

```
CREATE OR REPLACE MODEL `mydataset.arima_model`
  OPTIONS (
```

```
    model_type = 'ARIMA_PLUS',
    holiday_region = 'US',...) AS (
        training_data AS (SELECT * FROM `mydataset`.timeseries_data`),
        custom_holiday AS (
            SELECT
              'US' AS region,
              'members_day' AS holiday_name,
              primary_date,
              2 AS preholiday_days,
              2 AS postholiday_days
            FROM
              UNNEST(
                [
                  DATE('2016-06-15'),
                  DATE('2017-06-07'),
                  DATE('2018-06-06')])
                AS primary_date
          )
      );
```

Model only custom holidays

To model only custom holidays, don't specify a value for the **`HOLIDAY_REGION`** option, and provide the new holiday metadata in the **`holiday_statement`** argument.

The following example models only the custom holiday **`members_day`** for the **`US`** holiday region:

```
CREATE OR REPLACE MODEL `mydataset.arima_model`
  OPTIONS (
    model_type = 'ARIMA_PLUS',
    -- Don't specify HOLIDAY_REGION
    ...) AS (
        training_data AS (SELECT * FROM `mydataset.timeseries_data`),
        custom_holiday AS (
            SELECT
              'US' AS region,
              'members_day' AS holiday_name,
              primary_date,
              2 AS preholiday_days,
              2 AS postholiday_days
            FROM
              UNNEST(
                [
```

```
                DATE('2016-06-15'),
                DATE('2017-06-07'),
                DATE('2018-06-06')])
              AS primary_date
        )
    );
```

Change the metadata for built-in holidays

You can change the primary date and holiday effect window used by the model for one or more built-in holidays. To do this, specify the target holiday region with the `HOLIDAY_REGION` option, and then provide the modified holiday metadata in the `holiday_statement` argument.

The following example models all built-in holidays for the `US` holiday region, but models 3 years of the `EasterMonday` holiday with a 3-day holiday effect window instead of the default 9-day holiday effect window:

```
OPTIONS (
  model_type = 'ARIMA_PLUS',
  holiday_region = 'US',...) AS (
      training_data AS (SELECT * FROM `mydataset.timeseries_data`),
      custom_holiday AS (
          SELECT
            'US' AS region,
            'EasterMonday' AS holiday_name,
            primary_date,
            1 AS preholiday_days,
            1 AS postholiday_days
          FROM
            UNNEST(
              [
                DATE('2016-03-28'),
                DATE('2017-04-17'),
                DATE('2018-04-02')])
              AS primary_date
        )
    );
```

Model a subset of built-in holidays

To model only a subset of built-in holidays, don't specify a value for the `HOLIDAY_REGION` option, and provide a query based on the `bigquery-public-data.ml_datasets.holidays_and_events_for_forecasting` public table to specify the set of holidays to model.

The following example models all built-in holidays for the `US` holiday region except for the `Christmas` and `NewYears` holidays:

```
CREATE OR REPLACE MODEL `mydataset.arima_model`
  OPTIONS (
    model_type = 'ARIMA_PLUS',
    -- Don't specify HOLIDAY_REGION
    ...) AS (
        training_data AS (SELECT * FROM `mydataset.timeseries_data`),
        custom_holiday AS (
            SELECT *
            FROM `bigquery-public-data.ml_datasets.holiday`
            WHERE
              region = 'US'
              AND (holiday_name != 'Christmas' OR holiday_name != 'NewYear')
          )
      );
```

Custom holiday limitations

- Custom holiday modeling only works for models that have a `data_frequency` value of either `DAILY` or `AUTO_FREQUENCY`. If you use `AUTO_FREQUENCY`, the actual frequency of the time series data needs to be daily.
- You can't use the `TRANSFORM` clause (/bigquery/docs/reference/standard-sql/bigqueryml-syntax-create#transform) of the `CREATE MODEL` statement if you are performing custom holiday modeling.
- `holiday_name` column cannot have more than 500 distinct values.
- Custom holiday modeling uses an algorithm that automatically detects the significance of the holiday effect within the provided holiday effect window, and only extracts the holiday effect on the days that the algorithm classifies as significant. For example, if `primary date` is `01/02` with `preholiday_days` and `postholiday_days` set to `1`, the algorithm analyzes the holiday effect for these three days: `[01/01,`

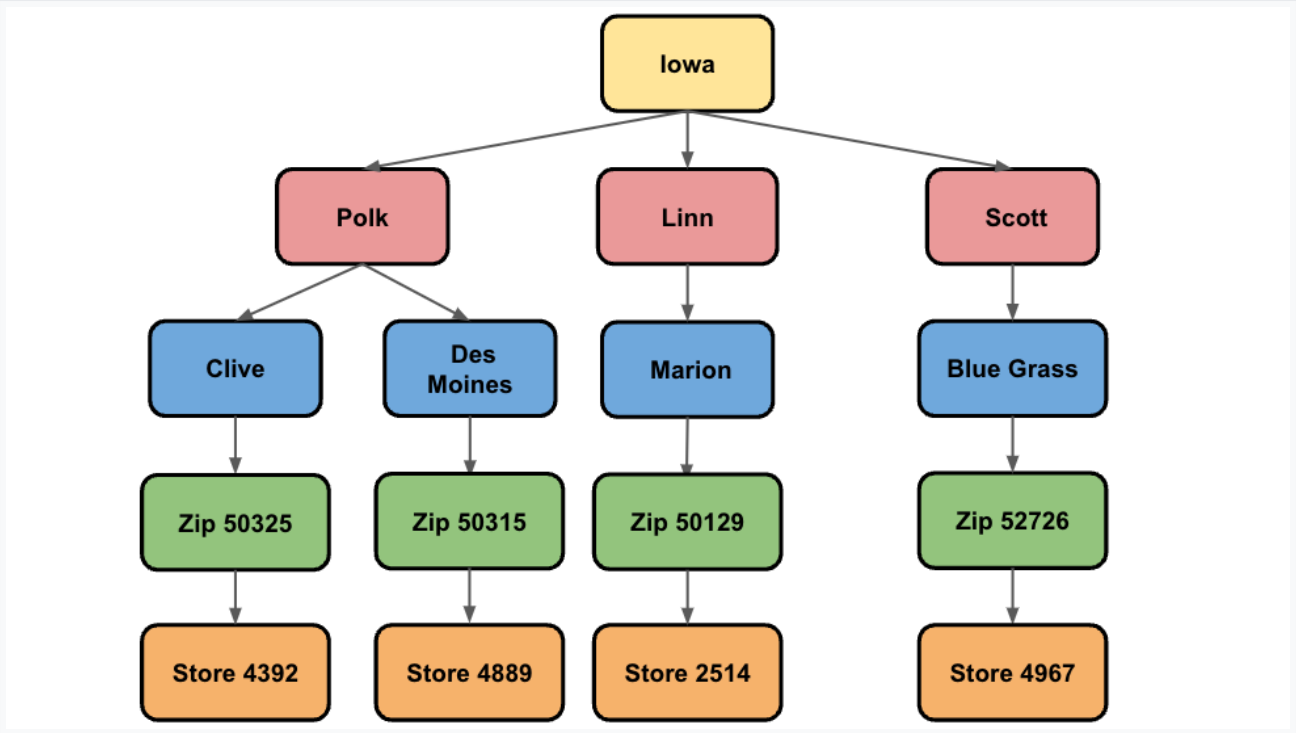
`01/02, 01/03]`. In the `ML.EXPLAIN_FORECAST` output, it is not guaranteed that all three of these days will have a holiday effect. Only those days within this window that have a significant holiday effect are associated with a non-zero holiday effect in the output.

- To get a better result from custom holiday modeling, don't specify the same holiday more than twice a year.
- If you specify the same holiday more than once, make sure its occurrences don't overlap. For example, a holiday that happens twice a year, with the first occurrence from December 1 - December 5 and the second occurrence from December 4 - December 8, won't work because these two occurrences overlap with each other on December 4 and December 5.
- Different holidays can have partial overlap with each other, but full overlap isn't allowed. For example, if holiday A has an occurrence from December 1 - December 5, then holiday B can have an occurrence from December 4 - December 8. However, holiday B can't have an occurrence from December 2 - December 4, which is fully within holiday A's occurrence, and holiday B can't have an occurrence from December 1 - December 8, which overlaps with all of holiday A's occurrence.
- In cases where holidays overlap for a period, the overlap between the holidays can't be identical in multiple years. If it is, then it isn't possible to determine each holiday's effect on the overlapped period. For example, suppose holiday A and holiday B overlap from December 2 - December 4 in 2024, and the time series has three years of data from 2022-2024. Holidays A and B can't overlap from December 2 - December 4 in 2023 and 2022 as well if you want to be able to determine the holiday effect for each.

Hierarchical reconciliation

You can break down or aggregate time series forecasts by specifying different dimensions of interest. These are known as hierarchical time series. For example, census data that reveals the total population per state can be broken down by city and zip code. Conversely, you can aggregate that data for each country or continent.

There are several techniques that you can use to generate and reconcile hierarchical forecasts. Take the following example, which shows a simplified hierarchical structure for liquor sales in the state of Iowa:



The lowest level shows the store level, followed by the zip code level, city, county, and finally by state. The goal for hierarchical forecasts is to make sure that all forecasts across each level reconcile. For example, given the earlier figure, this means that the forecast values for the cities of Clive and Des Moines must add up to the forecast value for Polk county. Similarly, the forecasts in Polk, Linn, and Scott must add up to the forecast in Iowa.

A common technique that you can use to generate reconciled forecasts is known as the *Bottom-Up* approach. In this approach, the forecasts are generated at the bottom level of the hierarchy first, before summing up the other levels. Taking the earlier example, the forecasts for each store are used to build the forecasting models for the other levels, so the store models are used to build the zip code models, the zip code models are used to build the city models, and so forth. In BigQuery ML, you use the `TIME_SERIES_ID_COL` (#time_series_id_col) option to identify the dimensions that you want to generate time series for, and the `HIERACHICAL_TIME_SERIES_ID_COLS` (#hierachical_time_series_id_cols) option to identify the dimensions that you want to roll up and reconcile. To learn more about generating a hierarchical time series, see Forecast multiple hierarchical time series with a univariate model (/bigquery/docs/arima-time-series-forecasting-with-hierarchical-time-series).

Locations

For information about supported locations, see Locations for non-remote models (/bigquery/docs/locations#locations-for-non-remote-models).

Limitations

`ARIMA_PLUS` models have the following limitations:

- For the input time series, the minimum length is 3 time points. The maximum length is 500,000 time points when the `DECOMPOSE_TIME_SERIES` option (/bigquery/docs/reference/standard-sql/bigqueryml-syntax-create-time-series#decompose_time_series) is set `TRUE` and 1,000,000 when the option is set to `FALSE`. When forecasting multiple time series at the same time, the limit applies to each time series.
- The maximum number of time series to forecast simultaneously using the ID columns is 100,000,000.
- When forecasting multiple time series simultaneously using the ID column, any invalid time series that fail the model fitting are ignored and don't appear in the results of forecast. For example, a single point time series. A warning message is shown in this case, and you can use the `ML.ARIMA_EVALUATE` function (/bigquery/docs/reference/standard-sql/bigqueryml-syntax-arima-evaluate) to retrieve the error message.
- The maximum time points to forecast is 10,000.
- Holiday effect modeling is effective only for approximately 5 years.
- After a multiple time series model is trained, the evaluation tab in the BigQuery page on the Google Cloud console only shows the evaluation metrics for the first 100 time series. To see the evaluation metrics for all of the time series, use the `ML.ARIMA_EVALUATE` function (/bigquery/docs/reference/standard-sql/bigqueryml-syntax-arima-evaluate).
- You can't export `ARIMA_PLUS` models.

Examples

The following examples show how to create different types of `ARIMA_PLUS` time series models.

Forecast a single time series

This example shows how to create a time series model that forecasts a single time series:

```
CREATE MODEL `project_id.mydataset.mymodel`
 OPTIONS(MODEL_TYPE='ARIMA_PLUS',
         time_series_timestamp_col='date',
         time_series_data_col='transaction') AS
SELECT
  date,
  transaction
FROM
  `mydataset.mytable`
```

Forecast multiple time series

This example shows how to create multiple time series models, one for each input time series:

```
CREATE MODEL `project_id.mydataset.mymodel`
 OPTIONS(MODEL_TYPE='ARIMA_PLUS',
         time_series_timestamp_col='date',
         time_series_data_col='transaction',
         time_series_id_col='company_name') AS
SELECT
  date,
  transaction,
  company_name
FROM
  `mydataset.mytable`
```

Forecast multiple time series using multiple time series ID columns

This example shows how to create multiple time series models for multiple IDs:

```
CREATE MODEL `project_id.mydataset.mymodel`
 OPTIONS(MODEL_TYPE='ARIMA_PLUS',
         time_series_timestamp_col='date',
         time_series_data_col='transaction',
         time_series_id_col=['company_name', 'department_name']) AS
SELECT
  date,
```

```
  transaction,
  company_name,
  department_name
FROM
  `mydataset.mytable`
```

Forecast multiple time series more quickly by using a fraction of the time points

This example shows how to create multiple time series models while improving training speed by using the `TIME_SERIES_LENGTH_FRACTION` and `MIN_TIME_SERIES_LENGTH` options:

```
CREATE MODEL `project_id.mydataset.mymodel`
 OPTIONS(MODEL_TYPE='ARIMA_PLUS',
         time_series_timestamp_col='date',
         time_series_data_col='transaction',
         time_series_id_col=['company_name', 'department_name'],
         time_series_length_fraction=0.5,
         min_time_series_length=30) AS
SELECT
  date,
  transaction,
  company_name,
  department_name
FROM
  `mydataset.mytable`
```

Forecast multiple time series more quickly by defining a maximum number of time points

This example shows how to create multiple time series models while improving training speed by using `MAX_TIME_SERIES_LENGTH` option:

```
CREATE MODEL `project_id.mydataset.mymodel`
 OPTIONS(MODEL_TYPE='ARIMA_PLUS',
         time_series_timestamp_col='date',
         time_series_data_col='transaction',
         time_series_id_col=['company_name', 'department_name'],
```

```
        max_time_series_length=50) AS
SELECT
  date,
  transaction,
  company_name,
  department_name
FROM
  `mydataset.mytable`
```

What's next

- Try the following tutorials to learn more about creating time series models:
  - Perform single time series forecasting from Google Analytics data (/bigquery/docs/arima-single-time-series-forecasting-tutorial)
  - Perform multiple time series forecasting with a single query from NYC Citi Bike trips data (/bigquery/docs/arima-multiple-time-series-forecasting-tutorial)
  - Scalable forecasting with millions of time series in BigQuery (/bigquery/docs/arima-speed-up-tutorial)
  - Use custom holidays in a time series forecasting model (/bigquery/docs/time-series-forecasting-holidays-tutorial)
  - Limit forecasted values for a time series model (/bigquery/docs/arima-time-series-forecasting-with-limits-tutorial)
  - Hierarchical time series forecasting (/bigquery/docs/arima-time-series-forecasting-with-hierarchical-time-series)
- Explore a notebook solution that helps you build a time series demand forecasting model (https://github.com/GoogleCloudPlatform/analytics-componentized-patterns/tree/master/retail/time-series/bqml-demand-forecasting) .

Except as otherwise noted, the content of this page is licensed under the Creative Commons Attribution 4.0 License (https://creativecommons.org/licenses/by/4.0/), and code samples are licensed under the Apache 2.0 License (https://www.apache.org/licenses/LICENSE-2.0). For details, see the Google Developers Site Policies (https://developers.google.com/site-policies). Java is a registered trademark of Oracle and/or its affiliates.

Last updated 2026-02-19 UTC.